SIR PADAMPAT SINGHANIA UNIVERSITY

WEB DEVELOPMENT / PROJECT REPORT

E-Library

A Smart Digital Library and Online Reading Platform

Student name	Padam Kishore
Enrollment number	25CS003692
Programme	B.Tech CSE (AI-ML)
Semester	III
University	Sir Padampat Singhania University
Course	Web Development
Submission date	5 November 2026

Project overview

This report documents the existing E-Library application and a focused catalogue/report extension. The application combines public book discovery, browser-based PDF/EPUB reading, Google sign-in and administrator-controlled MySQL records. The extension adds an isolated React/Bootstrap catalogue and a PHP CSV exporter while preserving the established reading workflow.

Repository: github.com/padam421/E-LIBRARY

Live library: e-library-c9t.pages.dev

Primary stack: HTML5, CSS3, JavaScript, Node.js/Express and MySQL.
Supplementary module: React, Bootstrap and PHP catalogue reporting.

02 / WEB DEVELOPMENT

Problem statement and objectives

Students and readers often rely on scattered PDF/EPUB files, shared links and separate applications to locate and read learning materials. This makes it difficult to search resources, resume reading and maintain a consistent digital collection. Library administrators also need a reliable way to organize book metadata, control visibility and manage access. E-Library addresses these problems through a centralized, responsive web application that combines searchable book discovery, browser-based reading, secure sign-in and an administrator dashboard backed by a MySQL database.

Project objectives

1. Develop a responsive digital library for desktop and mobile browsers.

2. Enable book discovery by title, author, category and PDF/EPUB format.

3. Provide browser-based PDF and EPUB reading with navigation and saved progress.

4. Implement Google sign-in, authenticated sessions and role-based administrator access.

5. Support MySQL-backed book creation, viewing, updating, deletion and bulk import.

6. Apply the course technologies through a small React/Bootstrap catalogue and a supplementary PHP CSV report tool.

7. Test major workflows and maintain source code, SQL schema and setup instructions on GitHub.

Design rationale

Keep the established architecture and add a useful, limited module. React and Bootstrap improve tabular discovery; PHP supplies an offline metadata report. Adding these technologies does not require replacing the primary web backend.

03 / WEB DEVELOPMENT

Scope and requirements

Scope area	Functional requirement
Public discovery	Searchable book cards/details, categories, catalogue filters and preview entry points.
Reading	PDF/EPUB viewing, navigation and saved progress in the existing reader interfaces.
Administration	Role-protected book CRUD, bulk import, visibility management and audit logs.
Reporting	Printable catalogue pages and supplementary PHP CSV export of public metadata.
Existing optional modules	AI assistance, premium access, support payments and media integration.

Non-functional requirements

- Responsive page layouts, labelled controls and usable empty/loading/error states.
- Server-side access checks and input handling at existing API boundaries.
- No secrets in tracked source; environment-specific service credentials remain outside Git.
- Reusable MySQL schema, documented setup and version-controlled source.
- Cached public data, paginated rendering and locally served catalogue dependencies.

Execution requirements

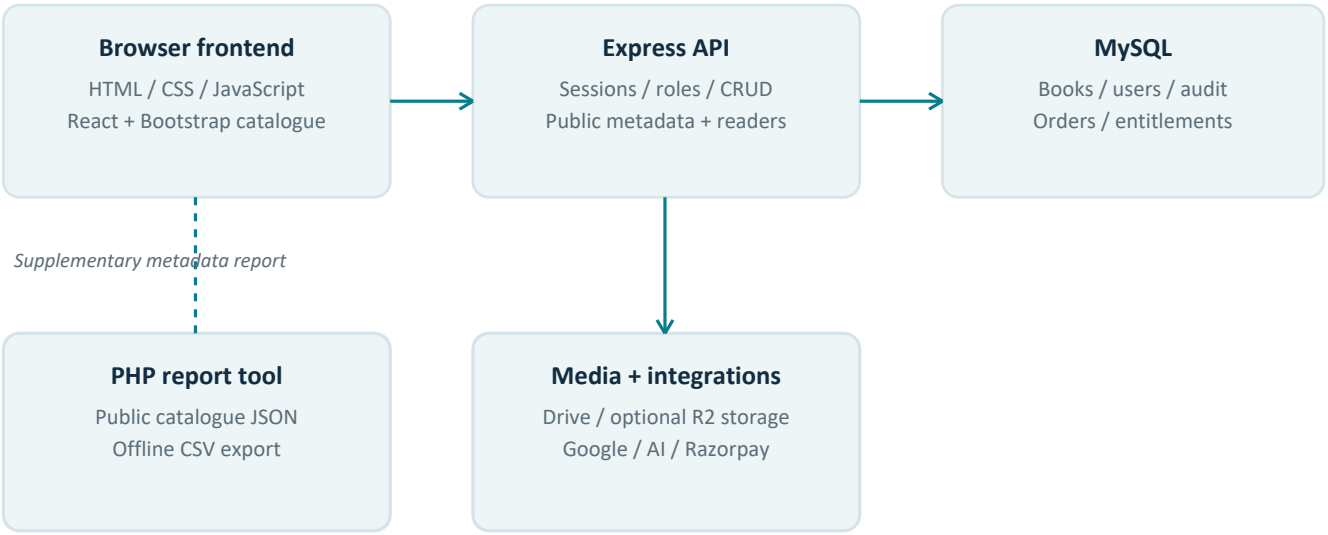
A modern browser, Node.js with npm, a MySQL database and the backend environment configuration are required for the core application. PHP 8.1+ is needed only for the supplementary exporter. Provider-backed sign-in, storage, AI and payments require their corresponding credentials.

Scope limits

This project is a digital reading platform rather than a physical circulation system. Physical issue/return tracking, barcode scanning and fine calculation are outside the current scope. No database migration was needed for the catalogue/report addition.

04 / WEB DEVELOPMENT

System design and architecture



Request flow

1. The browser loads the frontend and requests public metadata from GET /api/pdfs.

2. Express routes delegate to controllers and models; MySQL returns the book records.

3. The catalogue renders escaped text and applies search, category and format filters locally.

4. Reader endpoints apply session/access checks before serving protected assets.

5. Administrator mutations pass through role and CSRF middleware before database changes.

6. The PHP CLI exporter consumes public API JSON or a saved public snapshot and writes CSV.

Access boundaries

Google tokens are verified by the backend before signed sessions are used. Administrator routes check the session and database role. The new catalogue consumes the existing public metadata contract and introduces no alternate reader or administrator endpoint.

Deployment boundary

The static frontend can be served by Cloudflare Pages, with Express and MySQL hosted separately. PHP is a local/server-side reporting utility and is not executed by the static Pages deployment.

05 / WEB DEVELOPMENT

Database design

The core catalogue uses books_data and users. Existing extensions store premium rules, payment orders, user entitlements, support contributions and administrator activity. The diagram highlights selected entities; payment_settings is a separate configuration table.



Data group	Stored information
books_data	Book ID, title, author, category, description, asset references and is_private.
users	Unique email, Google profile fields, role and login timestamps.
Payments / access	Orders, entitlements, premium rules and supporter records.
Admin activity	Actor email, action, target information and timestamps.

SQL delivery: PDF-Library/sql/001_schema.sql, followed by the documented repair/audit/payment/support scripts. Parameterized database operations are used in the existing backend. The ER view distinguishes declared foreign keys from logical email/book references.

06 / WEB DEVELOPMENT

Implementation and technologies

Technology	Implementation role
HTML5	Semantic structure, forms, tables, book pages and reading interfaces.
CSS3	Responsive layouts, Flexbox/Grid, media queries, themes and print styles.
JavaScript / AJAX / JSON	DOM events, validation, Fetch requests and JSON client-server exchange.
React 18.3.1	Catalogue state, search/category/format filters, statistics and pagination.
Bootstrap 5.3.8	Catalogue grid, responsive table, forms, buttons and layout utilities.
Node.js / Express	Primary backend API, routes, middleware, authentication and business logic.
PHP 8.1+	Supplementary command-line conversion of public catalogue JSON to safe CSV.
MySQL	Book/user metadata, administrator audit records, payment records and CRUD.
Git / GitHub	Version control, repository sharing and automated syntax/PHP checks.

New module files

File / area	Implementation
frontend/catalogue.html	Standalone semantic page; a footer link makes it discoverable.
assets/js/catalogue.js	React state/effects, Fetch, filters, counts, pagination, retries and escaped text.
assets/css/catalogue.css	Catalogue-specific styles and print rules; Bootstrap remains isolated to this page.
php/export-catalogue.php	CLI-only JSON-to-CSV tool, bounded input, validation and safe output creation.
Tests and CI	Browser scenarios, 25 PHP checks, HTTP integration tests and CI coverage.

The existing reader, authentication, payment and database modules are retained. Local vendor files keep the catalogue independent of runtime CDN requests. PHP output includes only metadata fields; spreadsheet-sensitive values are escaped and existing output files are not overwritten.

07 / WEB DEVELOPMENT

Screenshots of major modules

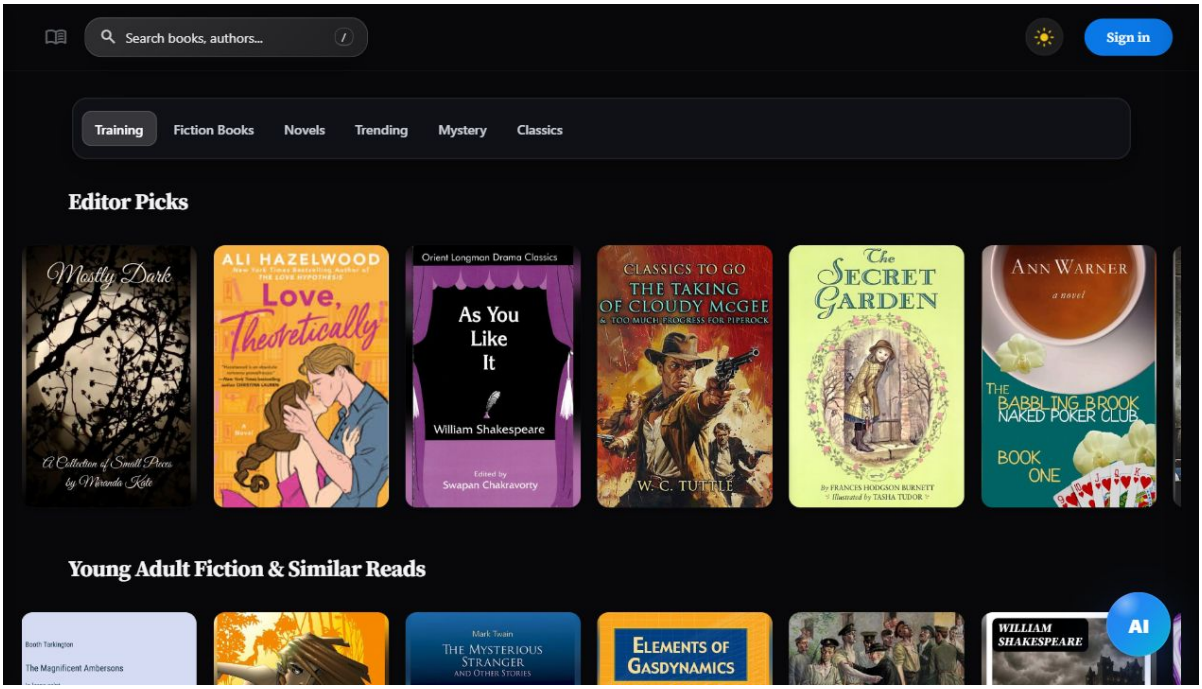


Figure 1. Existing public library discovery interface. Source: repository walkthrough screenshot.

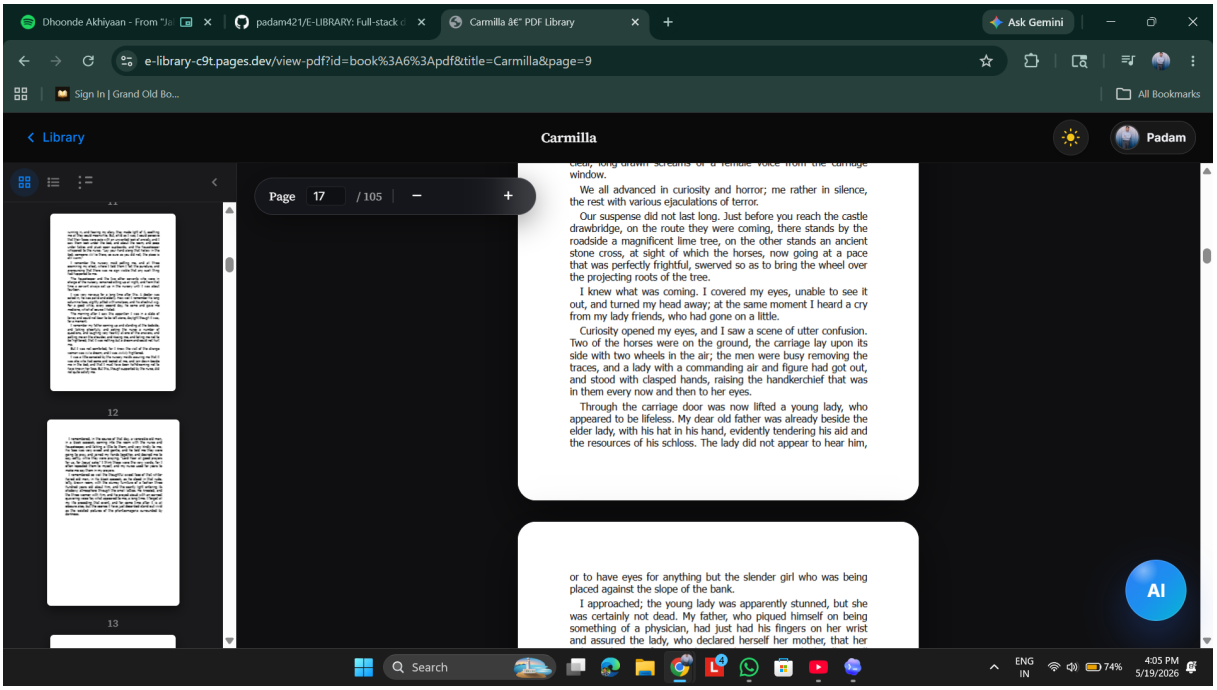


Figure 2. Existing PDF reading interface with navigation controls. Source: repository walkthrough screenshot.

These screenshots document the existing interface. They are not new end-to-end authentication, payment or reader test results.

07 / WEB DEVELOPMENT

Administration and catalogue

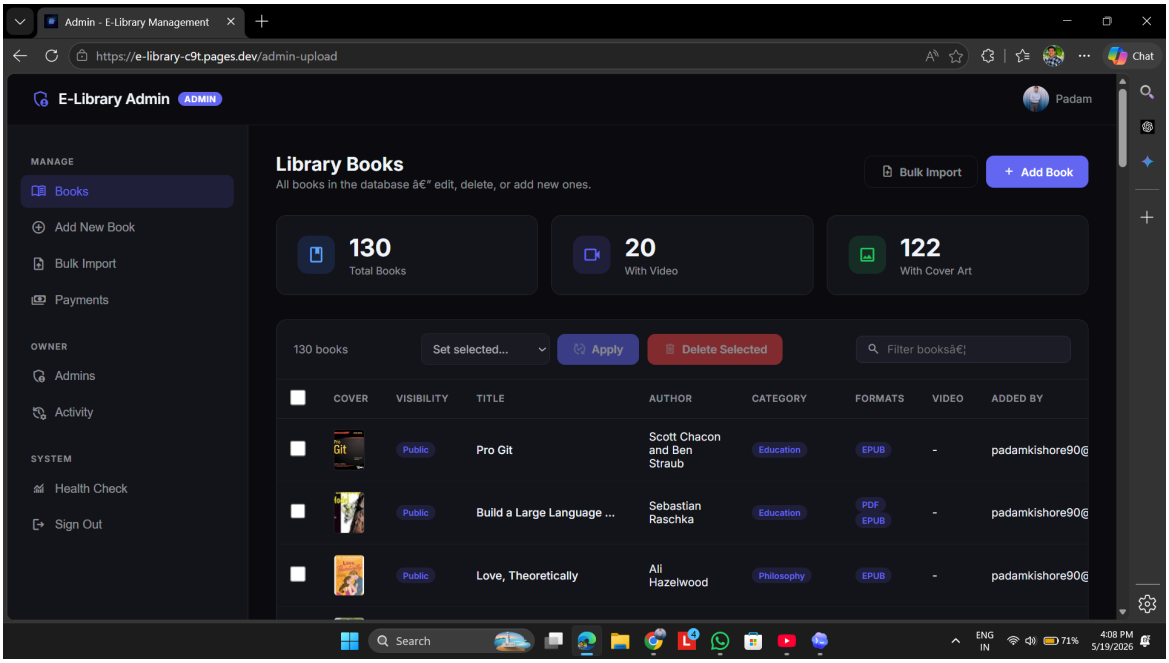


Figure 3. Existing administrator dashboard for book management. Source: repository walkthrough screenshot.

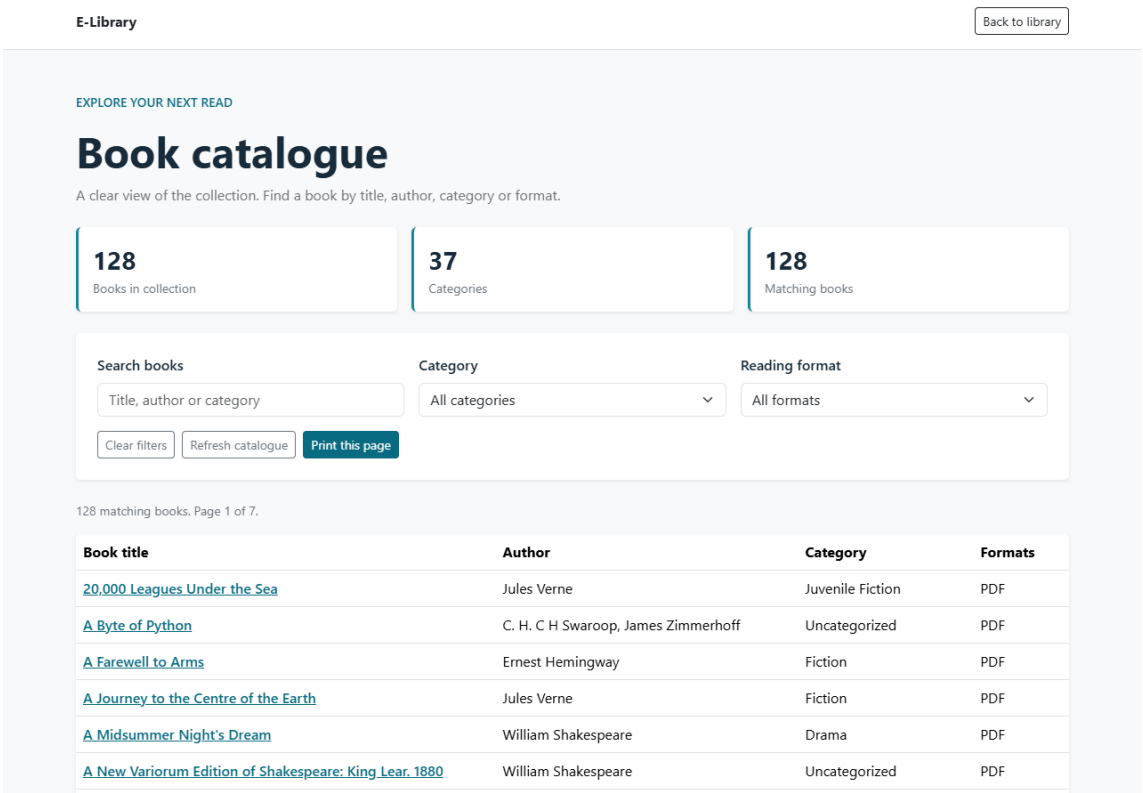


Figure 4. New React/Bootstrap catalogue captured locally using a 128-book database snapshot on 5 November 2026.

The catalogue reuses the public JSON endpoint, supports title/author/category/format filtering and renders at most 20 rows per page. Printing applies to the current page.

08 / WEB DEVELOPMENT

Testing and results

Validation was completed on 5 November 2026. Browser checks used deterministic fixtures for error and edge cases; the report screenshot and CSV check also used an actual local database snapshot.

Check	Result	Evidence / boundary
Backend syntax	PASS	npm run check completed.
Tracked secrets	PASS	No tracked secret files found; local credentials remain ignored.
Catalogue desktop/mobile	PASS	Rendering, filters, pagination, book links, print action and mobile overflow checks.
Catalogue resilience	PASS	Escaped malicious-looking title; API failure/retry, invalid response and empty catalogue.
PHP syntax / behaviour	PASS	Syntax valid; 25 checks for CSV, Unicode, formulas, privacy, invalid input and safe output.
PHP HTTP integration	PASS	Successful API export, upstream failure, redirect rejection and invalid response structure.
Actual CSV export	PASS	128 local catalogue records exported and verified as a five-column CSV.
Database / deployed API	PASS	Read-only database count: 128; deployed public API returned HTTP 200 with 130 books.
External full workflows	REVIEWED	OAuth, payments, AI and protected readers inspected in code; not fully exercised in this update.

Result interpretation

The tested catalogue and CSV reporting workflows passed. The two book counts refer to separate local and deployed snapshots and may change over time. Syntax checks and code inspection do not establish that every existing provider-backed workflow is operational.

Evaluation checks still required

Before a live demonstration, verify the deployed frontend/API configuration, sign in using the intended Google account, open a permitted PDF/EPUB and demonstrate administrator CRUD on suitable test records. Payment testing should use the appropriate configured test environment; no real purchase was made during this update.

09 / WEB DEVELOPMENT

Conclusion and future enhancements

E-Library combines discovery, reading and administrator-managed metadata in a single web application. The focused catalogue/report extension demonstrates React, Bootstrap and PHP alongside the existing HTML/CSS/JavaScript, Express and MySQL stack. The verified additions provide a usable filtered catalogue and a safe, portable metadata export without restructuring the core reading system.

Future enhancements

- Add server-side catalogue pagination and indexed search as the collection grows.

- Expand automated integration coverage for authentication, protected readers and administrator CRUD.

- Improve accessibility audits and offer additional interface languages.

- Extend reporting with scheduled summaries and richer collection analytics.

10 References

1. Project source code and SQL schema
https://github.com/padam421/E-LIBRARY

2. Project setup and architecture notes
https://github.com/padam421/E-LIBRARY/tree/main/docs

3. MDN Web Docs - HTML, CSS and JavaScript
https://developer.mozilla.org/en-US/docs/Web

4. React - createElement and browser rendering
https://react.dev/reference/react/createElement

5. Bootstrap 5.3 - getting started
https://getbootstrap.com/docs/5.3/getting-started/introduction/

6. Express - API reference
https://expressjs.com/en/4x/api.html

7. PHP - CSV output with fputcsv
https://www.php.net/manual/en/function.fputcsv.php

8. MySQL 8.0 Reference Manual
https://dev.mysql.com/doc/refman/8.0/en/

Academic requirements: the user-supplied Minor Project Guidelines and Google Form screenshots for the Web Development course. No public guideline URL was provided.